Ajedrez en Python: POO y Tkinter

Proyecto final de programación — Desarrollo de un juego de ajedrez completo aplicando Programación Orientada a Objetos e interfaz gráfica con Tkinter.

PEPE IGNACIO ALONSO

Made with GAMMA

Objetivos del Proyecto

Este proyecto combina teoría y práctica para consolidar los fundamentos del desarrollo de software moderno.

Dominar la POO

Comprender y aplicar herencia, polimorfismo y encapsulación en un proyecto real y funcional.

Interfaz con Tkinter

Diseñar una GUI limpia e interactiva: tablero de 8×8, colores, símbolos Unicode y respuesta al usuario.

Trabajo en Equipo

Dividir el proyecto en tres capas bien definidas: Lógica, Interfaz y Control, asignadas a cada miembro.

Made with GAMMA

El Equipo: Reparto de Tareas

Cada miembro asumió una capa arquitectónica independiente, favoreciendo la modularidad y la especialización.

1

⚙️ Pepe

El Creador

Lógica base: clase abstracta `Pieza` y subclases con herencia y polimorfismo.

2

🎨 Ignacio

El Diseñador

Frontend con Tkinter: grid 8×8, colores del tablero y renderizado de símbolos.

3

🕹️ Alonso

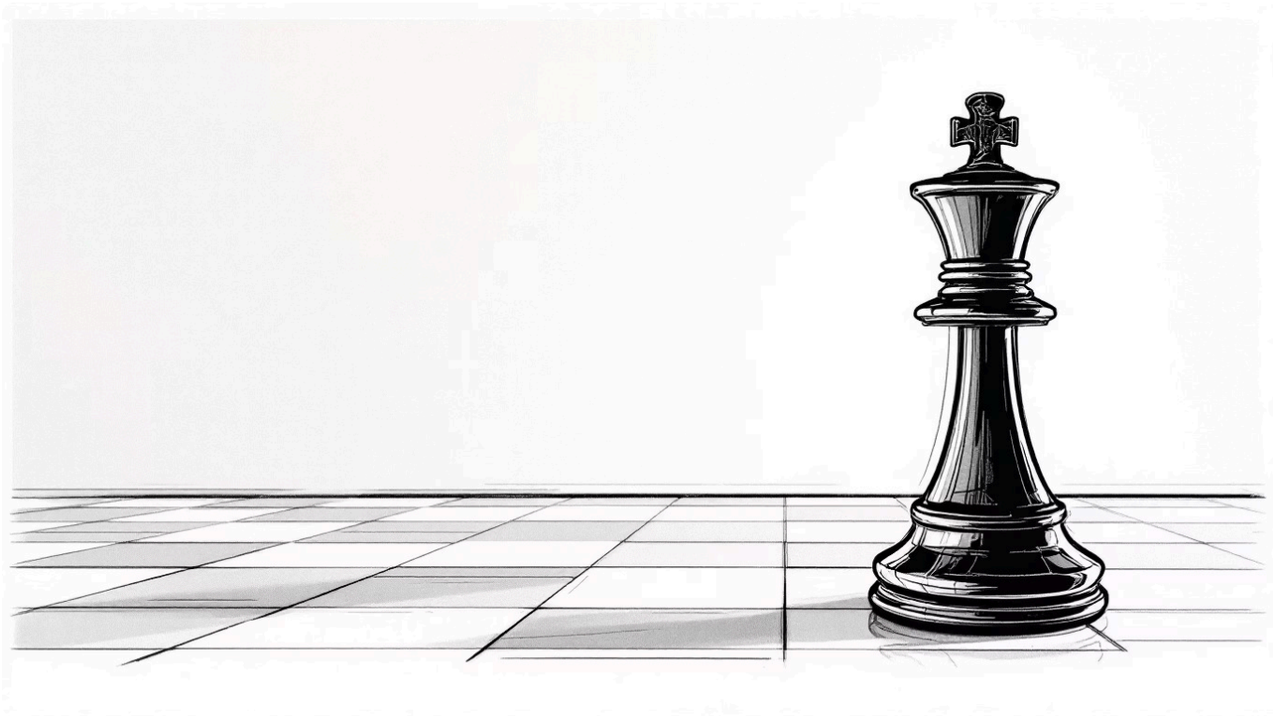
El Gestor

Controlador: sistema de clics, validación de turnos y actualización de la interfaz.

Made with GAMMA

El Motor del Juego: POO en Acción

PARTE DE PEPE



Arquitectura de clases

El núcleo del juego se apoya en una **Clase Base Abstracta (ABC)** llamada `Pieza`, que define la interfaz común para todas las piezas del tablero.

Herencia

Rey, Reina, Peón... heredan `color` y `posición`, y sobrescriben `mov_ok()` con su propia lógica matemática.

Polimorfismo

El controlador llama a `mov_ok()` sin conocer el tipo concreto de pieza: cada una responde a su manera.

Encapsulación

Los datos internos de cada pieza quedan protegidos, exponiéndose solo a través de métodos controlados.

Made with GAMMA

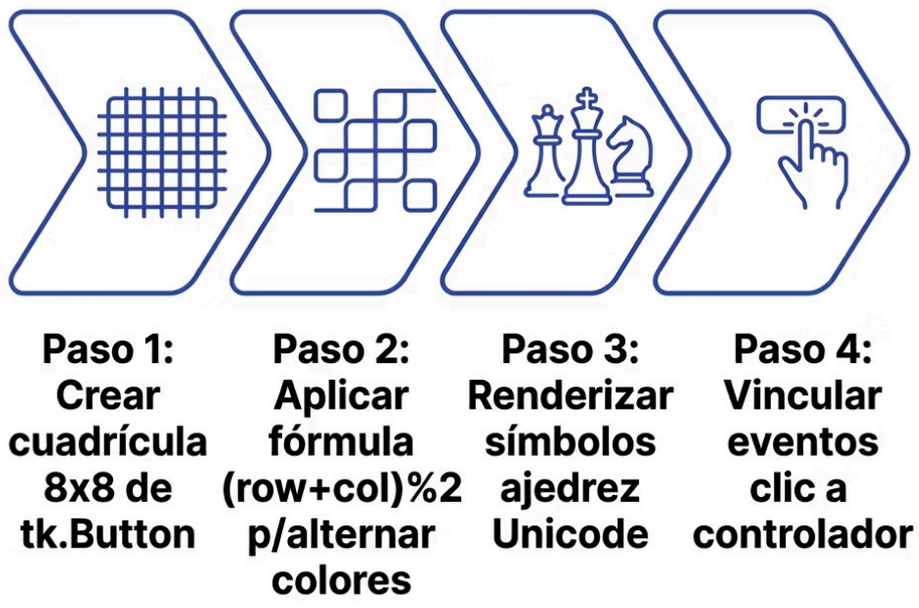
La Interfaz Gráfica: Tkinter

PARTE DE IGNACIO

Construcción del tablero visual

El tablero se genera mediante una matriz de `tk.Button` de 8×8, posicionados con el sistema de `grid` de Tkinter.

- **Colores alternos:** La fórmula `(fila + col) % 2 == 0` determina si la casilla es blanca o verde.
- **Símbolos Unicode:** Cada pieza se representa con su carácter estándar (♜, ♞, ♝...) directamente en el botón.
- **Separación de responsabilidades:** La capa visual no contiene lógica de movimientos ni de turnos.



Made with GAMMA

Interactividad y Control

PARTE DE ALONSO

1

1.er clic

El jugador selecciona la pieza de origen. La casilla se ilumina en **amarillo** como feedback visual.

2

Validación

Se comprueba que el turno corresponde al color correcto (blancas o negras) antes de permitir la selección.

3

2.º clic

El jugador elige el destino. Se llama a `mov_ok()` para validar el movimiento según las reglas de la pieza.

4

Victoria

Si se captura el Rey contrario, el sistema detecta la condición de victoria y finaliza la partida.

Made with GAMMA

Retos Superados

1 Sincronización matriz-botones

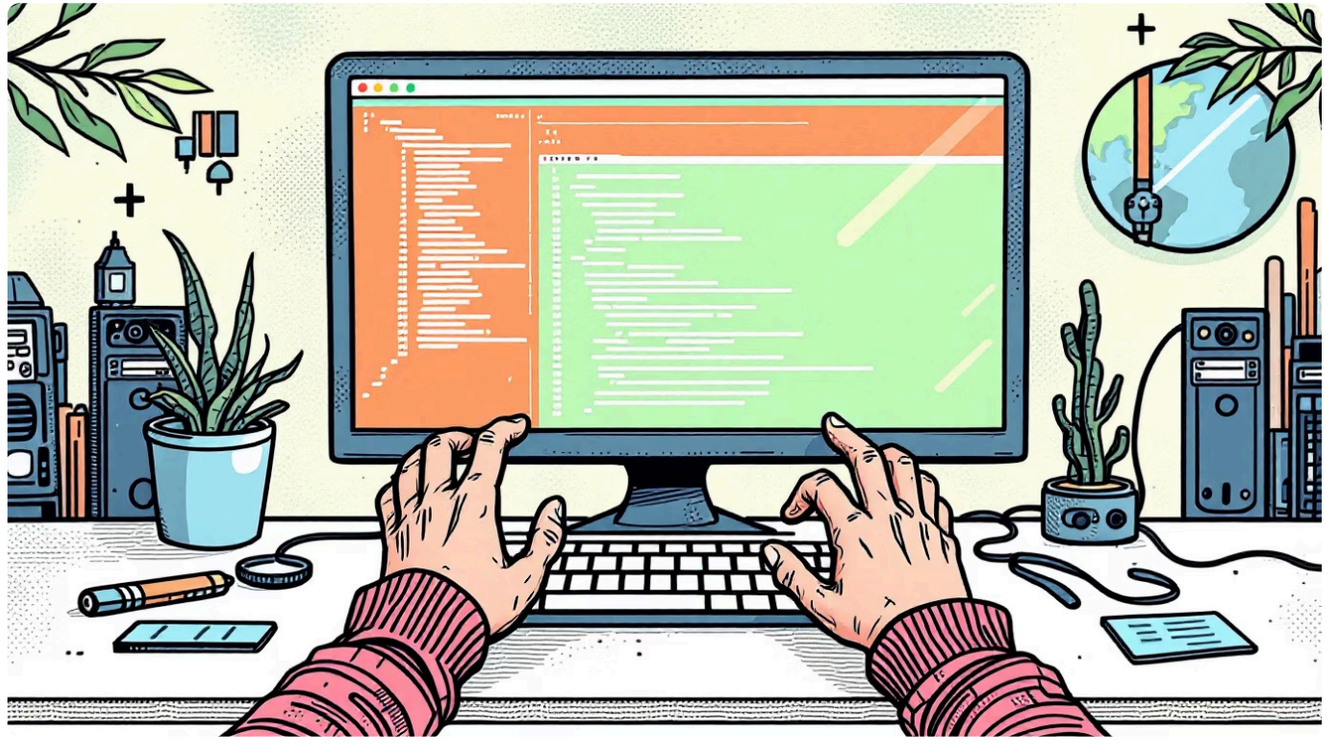
Conectar la matriz lógica de datos con la matriz visual de `tk.Button` fue el mayor desafío de arquitectura del proyecto.

2 Evitar el "fuego amigo"

Se implementó una validación explícita para impedir que un jugador capture sus propias piezas al seleccionar el destino.

3 Código limpio y modular

Mantener tres capas (Lógica, Vista, Control) independientes y bien comunicadas exigió disciplina y refactorización continua.



Made with GAMMA

Futuras Mejoras

El proyecto tiene una base sólida sobre la que seguir construyendo. Estas son las siguientes funcionalidades en el horizonte:

📜 Reglas Avanzadas

Implementar **enroque**, **coronación del peón** y **captura al paso** para completar el reglamento oficial.

🚧 Colisiones

Impedir que las piezas **salten por encima** de otras durante el movimiento, especialmente relevante para torres y alfiles.

⏱️ Temporizador

Añadir un **reloj de partida** por jugador, como en el ajedrez real, para introducir la presión del tiempo.

Made with GAMMA

Conclusión

> La POO no es solo un paradigma: es una forma de organizar el pensamiento. Este proyecto lo demuestra.

Hemos construido un juego de ajedrez funcional dividiendo un problema complejo en tres responsabilidades claras. Cada capa —Lógica, Interfaz y Control— trabaja de forma independiente pero coordinada, demostrando en la práctica los principios de la Programación Orientada a Objetos.

¿Preguntas? Abrimos el turno de debate para el profesor y la clase. 🎤

Made with GAMMA